

Multi-class Classification

Sourav Karmakar

Outline

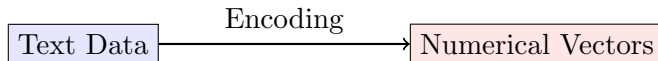
- 1 Encoding Categorical Data
- 2 Introduction to Multi-class Classification
- 3 One-vs-All Strategy
- 4 Categorical Cross Entropy Loss
- 5 The Softmax Function
- 6 Softmax and Sigmoid Similarity
- 7 Multinomial Logistic Regression or Softmax Regression
- 8 Evaluation Metrics

Data Preparation: Handling Categorical Features

Before we build models, we must convert text data into numbers.

Categorical Features Examples:

- Marital Status: [Unmarried, Married, Divorced, Widowed]
- Colour of a car: [Red, Blue, Grey, Black]
- Smoking habit: [Smoker, Non-Smoker]

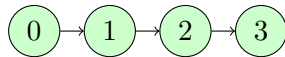


Technique 1: Label Encoding

Label Encoding assigns a unique integer to each category.

Example: Marital Status

- Unmarried $\rightarrow 0$
- Married $\rightarrow 1$
- Divorced $\rightarrow 2$
- Widowed $\rightarrow 3$

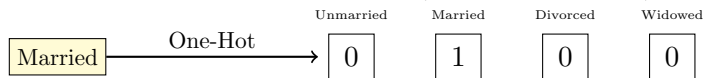


Implicit Order?

Limitation: The model might misunderstand the numbers as having an order (e.g., assuming Widowed $>$ Married), which is not true.

Technique 2: One-Hot Encoding

One-Hot Encoding creates binary columns (dummy variables) for each category.



Original	MS_Unmarried	MS_Married	MS_Divorced	MS_Widowed
Unmarried	1	0	0	0
Divorced	0	0	1	0
Married	0	1	0	0
Widowed	0	0	0	1

What is Multi-class Classification?

Binary Classification: Target has 2 classes (Yes/No, 0/1).

Multi-class Classification: Target has more than 2 classes ($C \geq 3$). **Real World**

Examples:

- ① **Email Foldering:** Classifying emails into Work, Friend, Family, Hobby.
- ② **Weather Prediction:** Sunny, Rainy, Overcast, Snowing.
- ③ **Medical Diagnosis:** Not Ill, Disease Type A, Disease Type B.
- ④ **Music Genre:** Pop, Rock, Classical, Jazz.

and many more ...

The IRIS Dataset: A Classic Example

Introduced by British statistician and biologist **Sir Ronald A. Fisher** (1936).



The Goal: Classify flowers into 3 species based on measurements.

Input Features (4 measurements):

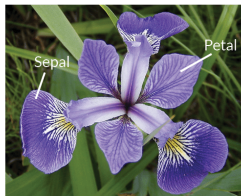
- Sepal Length
- Sepal Width
- Petal Length
- Petal Width

Target Classes (Species):

- 1 Iris Setosa (0)
- 2 Iris Versicolor (1)
- 3 Iris Virginica (2)

This dataset is the “Hello World” of Multi-class classification.

The IRIS Dataset: A Classic Example



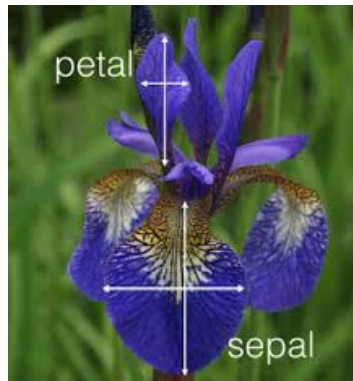
Iris Versicolor



Iris Setosa



Iris Virginica

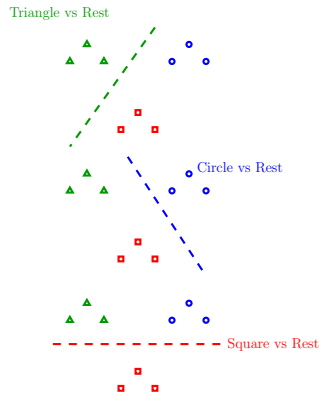


One-vs-All (One-vs-Rest) Model

How do we use binary classifiers for multi-class problems?

Strategy: If we have C classes, we train C separate binary classifiers.

- **Classifier 1:** Class 1 (Positive) vs All Others (Negative)
- **Classifier 2:** Class 2 (Positive) vs All Others (Negative)
- **Classifier 3:** Class 3 (Positive) vs All Others (Negative)



Prediction:

$$\text{Predicted Class} = \max_k (h_{\theta}^{(k)}(x))$$

Drawbacks of One-vs-All Strategy

While intuitive, this strategy introduces significant theoretical issues:

- **1. Severe Class Imbalance:** For a problem with C classes, each binary classifier is trained on a ratio of roughly $1 : (C - 1)$. The “Negative” class (Rest) is usually much larger than the “Positive” class, which can bias the model to simply predict “Negative” more often.
- **2. Uncalibrated Scales:** The C classifiers are trained independently. The raw scores (or probabilities) they output may not be directly comparable.
 - *Example:* Classifier A might output values in range $[0.1, 0.4]$ while Classifier B outputs in $[0.8, 0.99]$. Picking the “max” score might just favor the “loudest” model, not the most accurate one.
- **3. Ambiguity:** Regions exist in the feature space where multiple classifiers predict “1” (Positive) simultaneously, or where all predict “0” (Negative), leaving the final class undefined without arbitrary tie-breaking rules.

Categorical Cross Entropy Loss (CCE)

In multi-class settings, we compare the **Predicted probability distribution** against the **True distribution (One-Hot Encoded)**. The Formula for a single instance (i):

$$Loss^{(i)} = - \sum_{k=1}^C y_k^{(i)} \log(\hat{y}_k^{(i)})$$

Where:

- y_k : Actual label (0 or 1, one-hot encoded)
- $\hat{y}_k$: Predicted probability for class k
- C : Number of classes

Total Cost Function (Average over m examples):

$$J(\theta) = \frac{1}{m} \sum_{i=1}^m Loss^{(i)} = -\frac{1}{m} \sum_{i=1}^m \sum_{k=1}^C y_k^{(i)} \log(\hat{y}_k^{(i)})$$

Numerical Example: CCE Calculation

Let's calculate the loss for one flower.

Scenario: We have 3 classes. The flower is actually Class 2.

Step 1: Identify Actual Values (y) Since it is Class 2, the One-Hot vector is:

$$y = [0, \quad 1, \quad 0]$$

Numerical Example: CCE Calculation

Let's calculate the loss for one flower.

Scenario: We have 3 classes. The flower is actually Class 2.

Step 1: Identify Actual Values (y) Since it is Class 2, the One-Hot vector is:

$$y = [0, \quad 1, \quad 0]$$

Step 2: Identify Predicted Probabilities ($\hat{y}$) Suppose our model predicts:

$$\hat{y} = [0.2, \quad 0.7, \quad 0.1]$$

Numerical Example: CCE Calculation

Step 3: Apply Formula

recall: $Loss = - \sum (y_k \cdot \log(\hat{y}_k))$

Numerical Example: CCE Calculation

Step 3: Apply Formula

recall: $Loss = -\sum(y_k \cdot \log(\hat{y}_k))$

$$Loss = -[(y_1 \cdot \log(\hat{y}_1)) + (y_2 \cdot \log(\hat{y}_2)) + (y_3 \cdot \log(\hat{y}_3))]$$

Numerical Example: CCE Calculation

Step 3: Apply Formula

recall: $Loss = -\sum(y_k \cdot \log(\hat{y}_k))$

$$Loss = -[(y_1 \cdot \log(\hat{y}_1)) + (y_2 \cdot \log(\hat{y}_2)) + (y_3 \cdot \log(\hat{y}_3))]$$

Substituting the values:

$$Loss = -[(0 \cdot \log(0.2)) + (1 \cdot \log(0.7)) + (0 \cdot \log(0.1))]$$

Numerical Example: CCE Calculation

Step 3: Apply Formula

recall: $Loss = -\sum(y_k \cdot \log(\hat{y}_k))$

$$Loss = -[(y_1 \cdot \log(\hat{y}_1)) + (y_2 \cdot \log(\hat{y}_2)) + (y_3 \cdot \log(\hat{y}_3))]$$

Substituting the values:

$$Loss = -[(0 \cdot \log(0.2)) + (1 \cdot \log(0.7)) + (0 \cdot \log(0.1))]$$

Simplifying (terms with 0 vanish):

$$Loss = -[0 + \log(0.7) + 0]$$

Numerical Example: CCE Calculation

Step 3: Apply Formula

recall: $Loss = -\sum(y_k \cdot \log(\hat{y}_k))$

$$Loss = -[(y_1 \cdot \log(\hat{y}_1)) + (y_2 \cdot \log(\hat{y}_2)) + (y_3 \cdot \log(\hat{y}_3))]$$

Substituting the values:

$$Loss = -[(0 \cdot \log(0.2)) + (1 \cdot \log(0.7)) + (0 \cdot \log(0.1))]$$

Simplifying (terms with 0 vanish):

$$Loss = -[0 + \log(0.7) + 0]$$

Final Calculation:

$$Loss \approx -(-0.356) = \mathbf{0.356}$$

From Raw Scores to Probabilities

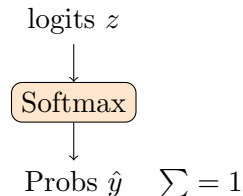
Our linear models produce raw scores (logits) z , which can be positive, negative, or large numbers.

Problem: We need probabilities that:

- 1 Are between 0 and 1.
- 2 Sum up to exactly 1 across all classes.

Solution: The Softmax Function

$$\hat{y}_k = \sigma(z)_k = \frac{e^{z_k}}{\sum_{j=1}^C e^{z_j}}$$



Softmax Calculation Example

Step 1: Raw Logits (z) Suppose the model outputs scores for 3 classes:

$$z = [2.0, \quad 1.0, \quad 0.1]$$

Softmax Calculation Example

Step 1: Raw Logits (z) Suppose the model outputs scores for 3 classes:

$$z = [2.0, \quad 1.0, \quad 0.1]$$

Step 2: Exponentiate (e^z)

$$e^{2.0} \approx 7.39$$

$$e^{1.0} \approx 2.72$$

$$e^{0.1} \approx 1.10$$

Softmax Calculation Example

Step 1: Raw Logits (z) Suppose the model outputs scores for 3 classes:

$$z = [2.0, \quad 1.0, \quad 0.1]$$

Step 2: Exponentiate (e^z)

$$e^{2.0} \approx 7.39$$

$$e^{1.0} \approx 2.72$$

$$e^{0.1} \approx 1.10$$

Step 3: Sum of Exponentials

$$\sum = 7.39 + 2.72 + 1.10 = 11.21$$

Softmax Calculation Example

Step 4: Normalize (Divide by Sum)

We divide each exponential by the sum (11.21) to get probabilities:

$$P(\text{Class 1}) = 7.39/11.21 \approx \mathbf{0.66}$$

$$P(\text{Class 2}) = 2.72/11.21 \approx \mathbf{0.24}$$

$$P(\text{Class 3}) = 1.10/11.21 \approx \mathbf{0.10}$$

Result sums to $0.66 + 0.24 + 0.10 = 1.0$.

Why Exponential Function?

Why do we use e^z in Softmax? Why not just square z or use absolute values?

- ① **Non-Negativity:** Probabilities cannot be negative. Even if the raw score z is negative (e.g., -5), e^{-5} is positive.
- ② **Differentiability:** The exponential function is smooth and easy to differentiate, which is vital for Gradient Descent.
- ③ **Emphasis on Max:** The exponential function “exaggerates” differences. A slightly higher score gets a much higher probability, pushing the model to be decisive.

Softmax is Generalised Sigmoid

What happens if we apply Softmax to a **binary** classification ($C = 2$)? Let raw scores be z_1 and z_2 . The probability of Class 1 is:

$$\sigma(z)_1 = \frac{e^{z_1}}{e^{z_1} + e^{z_2}}$$

Softmax is Generalised Sigmoid

What happens if we apply Softmax to a **binary** classification ($C = 2$)? Let raw scores be z_1 and z_2 . The probability of Class 1 is:

$$\sigma(z)_1 = \frac{e^{z_1}}{e^{z_1} + e^{z_2}}$$

Let's divide numerator and denominator by e^{z_1} :

$$\sigma(z)_1 = \frac{1}{1 + \frac{e^{z_2}}{e^{z_1}}} = \frac{1}{1 + e^{(z_2 - z_1)}}$$

Softmax is Generalised Sigmoid

What happens if we apply Softmax to a **binary** classification ($C = 2$)? Let raw scores be z_1 and z_2 . The probability of Class 1 is:

$$\sigma(z)_1 = \frac{e^{z_1}}{e^{z_1} + e^{z_2}}$$

Let's divide numerator and denominator by e^{z_1} :

$$\sigma(z)_1 = \frac{1}{1 + \frac{e^{z_2}}{e^{z_1}}} = \frac{1}{1 + e^{(z_2 - z_1)}}$$

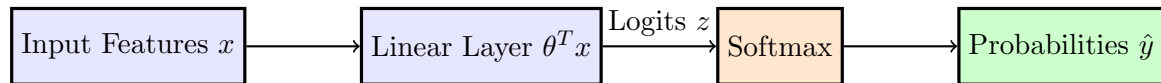
If we define the relative score difference as $z = z_1 - z_2$, this becomes:

$$\frac{1}{1 + e^{-z}}$$

Conclusion: This is exactly the **Sigmoid** function. Softmax is simply Sigmoid generalized for more than 2 classes.

Putting it all together: Softmax Regression

Also known as **Multinomial Logistic Regression** or **Maximum Entropy Classifier**.
The Architecture:



- 1 Calculate linear scores (Logits) for each class.
- 2 Apply Softmax to get probabilities.
- 3 Calculate Cross Entropy Loss against true labels.
- 4 Update weights using Gradient Descent.

The Confusion Matrix (M_C)

How do we evaluate a multi-class model? Accuracy alone is not enough. We use the **Confusion Matrix** of size $|C| \times |C|$.

		Predicted Class (j)		
Actual Class (i)		m_{11}	m_{12}	m_{13}
		m_{21}	m_{22}	m_{23}
		m_{31}	m_{32}	m_{33}

Understanding the Matrix:

- **Diagonal (m_{ii}):**
Correct Predictions
(True Positives for class i).
- **Off-Diagonal (m_{ij}):**
Errors
(Class i misclassified as j).

Metric Formulas

Based on the matrix M_C , we define:

1. Accuracy (Overall)

$$\text{Accuracy} = \frac{\text{Sum of Diagonal}}{\text{Total Sum}} = \frac{\text{trace}(M_C)}{\text{sum}(M_C)}$$

2. Precision (Per Class i) “Of all instances predicted as i , how many were actually i ?”

$$\alpha_i = \frac{m_{ii}}{\sum_k m_{ki}} = \frac{\text{Diagonal Element}}{\text{Sum of Column } i}$$

3. Recall (Per Class i) “Of all actual instances of i , how many did we predict correctly?”

$$\beta_i = \frac{m_{ii}}{\sum_k m_{ik}} = \frac{\text{Diagonal Element}}{\text{Sum of Row } i}$$

Numerical Example: Accuracy Calculation

Let's evaluate a model trained on the IRIS dataset (3 classes). Following is the confusion matrix obtained on the test dataset.

The Confusion Matrix

		Predicted Class		
		Setosa	Versi	Virg
Actual Class	Setosa	10	2	0
	Versi	1	15	4
	Virg	0	3	15

Numerical Example: Accuracy Calculation

Let's evaluate a model trained on the IRIS dataset (3 classes). Following is the confusion matrix obtained on the test dataset.

The Confusion Matrix

		Predicted Class		
		Setosa	Versi	Virg
Actual Class	Setosa	10	2	0
	Versi	1	15	4
	Virg	0	3	15

- **Total Samples:**

$$(10 + 2 + 0) +$$

$$(1 + 15 + 4) +$$

$$(0 + 3 + 15) = \mathbf{50}$$

Numerical Example: Accuracy Calculation

Let's evaluate a model trained on the IRIS dataset (3 classes). Following is the confusion matrix obtained on the test dataset.

The Confusion Matrix

		Predicted Class		
		Setosa	Versi	Virg
Actual Class	Setosa	10	2	0
	Versi	1	15	4
	Virg	0	3	15

- **Total Samples:**
 $(10 + 2 + 0) +$
 $(1 + 15 + 4) +$
 $(0 + 3 + 15) = 50$
- **Correct (Trace):**
 $10 + 15 + 15 = 40$

Numerical Example: Accuracy Calculation

Let's evaluate a model trained on the IRIS dataset (3 classes). Following is the confusion matrix obtained on the test dataset.

The Confusion Matrix

		Predicted Class		
		Setosa	Versi	Virg
Actual Class	Setosa	10	2	0
	Versi	1	15	4
	Virg	0	3	15

- **Total Samples:**
 $(10 + 2 + 0) +$
 $(1 + 15 + 4) +$
 $(0 + 3 + 15) = 50$
- **Correct (Trace):**
 $10 + 15 + 15 = 40$
- **Accuracy (Overall):**
 $\frac{Trace}{Total} = \frac{40}{50} = 0.8$

Numerical Example: Precision & Recall for “Versicolor”

Focus on the middle class (**Versicolor**).

Matrix Row 2 (Actual): $\rightarrow \text{Sum} = 20$

Matrix Col 2 (Predicted): $\rightarrow \text{Sum} = 20$

Numerical Example: Precision & Recall for “Versicolor”

Focus on the middle class (**Versicolor**).

Matrix Row 2 (Actual): $\rightarrow \text{Sum} = 20$

Matrix Col 2 (Predicted): $\rightarrow \text{Sum} = 20$

Calculate Precision (Column Focus):

Numerical Example: Precision & Recall for “Versicolor”

Focus on the middle class (**Versicolor**).

Matrix Row 2 (Actual): $\rightarrow \text{Sum} = 20$

Matrix Col 2 (Predicted): $\rightarrow \text{Sum} = 20$

Calculate Precision (Column Focus):

$$\text{Precision}_{\text{Versi}} = \frac{\text{Correct Versi}}{\text{Total Predicted Versi}} = \frac{15}{2 + 15 + 3} = \frac{15}{20} = \mathbf{0.75}$$

Numerical Example: Precision & Recall for “Versicolor”

Focus on the middle class (**Versicolor**).

Matrix Row 2 (Actual): $\rightarrow$ Sum = 20

Matrix Col 2 (Predicted): $\rightarrow$ Sum = 20

Calculate Precision (Column Focus):

$$\text{Precision}_{\text{Versi}} = \frac{\text{Correct Versi}}{\text{Total Predicted Versi}} = \frac{15}{2 + 15 + 3} = \frac{15}{20} = \mathbf{0.75}$$

Calculate Recall (Row Focus):

Numerical Example: Precision & Recall for “Versicolor”

Focus on the middle class (**Versicolor**).

Matrix Row 2 (Actual): $\rightarrow \text{Sum} = 20$

Matrix Col 2 (Predicted): $\rightarrow \text{Sum} = 20$

Calculate Precision (Column Focus):

$$\text{Precision}_{\text{Versi}} = \frac{\text{Correct Versi}}{\text{Total Predicted Versi}} = \frac{15}{2 + 15 + 3} = \frac{15}{20} = \mathbf{0.75}$$

Calculate Recall (Row Focus):

$$\text{Recall}_{\text{Versi}} = \frac{\text{Correct Versi}}{\text{Total Actual Versi}} = \frac{15}{1 + 15 + 4} = \frac{15}{20} = \mathbf{0.75}$$

Numerical Example: Precision & Recall for “Versicolor”

Focus on the middle class (**Versicolor**).

Matrix Row 2 (Actual): $\rightarrow$ Sum = 20

Matrix Col 2 (Predicted): $\rightarrow$ Sum = 20

Calculate Precision (Column Focus):

$$\text{Precision}_{\text{Versi}} = \frac{\text{Correct Versi}}{\text{Total Predicted Versi}} = \frac{15}{2 + 15 + 3} = \frac{15}{20} = \mathbf{0.75}$$

Calculate Recall (Row Focus):

$$\text{Recall}_{\text{Versi}} = \frac{\text{Correct Versi}}{\text{Total Actual Versi}} = \frac{15}{1 + 15 + 4} = \frac{15}{20} = \mathbf{0.75}$$

Note: For Setosa (Row 1, Col 1): Precision = 10/11 = 0.90, Recall = 10/12 = 0.83.

- **Encoding:** Convert labels to numbers (Label) or vectors (One-Hot).
- **Model:** Softmax Regression generalizes Logistic Regression for $C > 2$.
- **Activation:** Softmax ensures outputs are valid probabilities.
- **Loss:** Categorical Cross Entropy penalizes wrong high-confidence predictions.
- **Evaluation:** Always check the Confusion Matrix to calculate Class-wise Precision and Recall.

Thank You